

**The Unreasonable Extensibility of Computation:
What a Fixed Machine Can Do, and When It Is Possible to
Know**

PIATRA . INSTITUTE

September 2026

Abstract

A handheld games console released in 2004 can now generate text locally. A published runtime runs a 90-million-parameter model on a Sony PSP with 64 MB of memory and a 333 MHz processor, at about 0.55 tokens per second, with no network connection. The hardware is unchanged; what changed is the available software and trained parameters. We treat capability as a relation among a machine, an artifact, a task, a budget and a quality threshold, so the set of tasks a fixed machine can perform depends on the date. Three results follow. First, arithmetic on the runtime’s published byte format shows that full-precision weights need 7.24 times the console’s weight cache and four-bit quantisation still leaves the model at 1.02 times the cache, so 0.87 MB per token must be streamed; of the sixteen combinations of the runtime’s four techniques, 2 reach 0.4 tokens per second, three techniques are individually necessary, and on the 32 MB revision of the console none runs. Second, in a model calibrated to a census of 113 algorithm families, between 0.83 and 0.89 of improvement events change no capability, and the burstiness of capability crossings relative to efficiency gains rises from 0.84 to 1.67 as the budget becomes more restrictive. Third, deployable capability requires every component of a software stack to remain runnable, so it saturates at 0.43 of known capability for a five-component stack; for such a stack to last as long as the mean interval between improvements in its field, each component would need a mean life of 201 years.

Introduction

A custom runtime executes Falcon-H1-Tiny-90M-Instruct, a 90-million-parameter instruction-tuned model whose 24 layers each run grouped-query attention and a Mamba2 recurrent branch in parallel, on the PSP’s 333 MHz MIPS processor, generating about 0.5 to 0.6 tokens per second on a PSP-3000 (Thatblend, 2026; Technology Innovation Institute, 2026; Gu and Dao, 2023). The weights are stored on the memory card and nothing is streamed from a server. Every property of the artifact stated here comes from the runtime’s documentation and the model card; the console’s release date serves only as context.

The runtime documents its requirements. Supported hardware is the PSP-2000, PSP-3000, Street or Go with custom firmware; “a PSP-1000 only has 32 MB of RAM and it won’t work”. Dense weights are stored in a block format of 32 weights and one 16-bit scale per 18 bytes, with rows independently addressable so that a single embedding row can be read without parsing the file. Norms, convolution weights and the state-space tensors remain at full precision, about 0.5 MB in total, and the tokenizer’s 32,768 pieces are embedded in the file. The file is 52 MB, which “doesn’t fit in a 64 MB console alongside ~8 MB of runtime state”, so the runtime allocates correctness-critical state first, caches 44 MB of heap plus a 4 MB volatile partition the firmware does not use, and “streams the last few MB from the memory stick per token”. The key-value cache is 8-bit with a scale per head, and the matrix-vector kernels run on the vector unit instead of the scalar floating-point unit. The author describes the output plainly: the model “can answer basic questions”, writes poems and short stories, produces code that does not work, and is “not really useful for anything”. The argument below depends only on the model running.

In one sense the console could always do this, because the running configuration was always a physically reachable state of the hardware. A statement in 2004 about what the console could do was, however, a statement about what anyone then knew how to make it do within a budget; the set of physically reachable configurations cannot be enumerated. That statement was true when made and is false now, while the machine is unchanged.

Senses of machine capability

Four senses of “what a machine can do” need to be distinguished.

Physical possibility is the set of state transitions the device supports. Loading software changes the state of the memory, so the hardware is not strictly unchanged between running a game and running an inference loop. Here *fixed hardware* means that the architecture, clock ceiling, memory and storage capacities, bandwidths, peripherals and power envelope are held constant while the loaded information varies. The set of physically reachable configurations does not change.

Computability in principle is the sense Turing settled. A universal machine can simulate any machine given its description (Turing, 1937), which is why a games console and an inference engine can be the same object. In this sense the answer was fixed in 2004, but the sense ignores resources: universality does not say whether an implementation fits in 48 MB, returns a token within two seconds or runs within a battery budget.

Resource-bounded feasibility changes over time. Hartmanis and Stearns (1965) indexed computability by the resources a machine uses, and most practical questions about what a computer can do are questions about budgets (Aaronson, 2013).

Operational capability adds a quality threshold, because a task performed badly enough is not performed. The console runtime illustrates why the two must be tracked separately: its documentation notes that cache size “only affects speed, greedy output is identical at any setting”, so the artifact moves along the resource axis without moving along the quality axis.

Write a task as an input space, an output space, a distribution over inputs and a performance measure. Let H be the fixed hardware, p an executable artifact including code, trained parameters, tables, indexes and the runtime, $Q(p, H, \tau)$ the task quality, $R(p, H, \tau)$ the vector of resources used, and B a budget vector. Let K_t be the set of artifacts that exist and can be obtained at time t . The feasible capability set is

$$\mathcal{C}_t(H, B, q) = \{ \tau : \exists p \in K_t, Q(p, H, \tau) \geq q, R(p, H, \tau) \preceq B \},$$

and the intrinsic set $\mathcal{C}^*$ replaces K_t with every artifact that can be instantiated on H . Then $\mathcal{C}_t \subseteq \mathcal{C}^*$, where the right-hand side is fixed and the left-hand side depends on the date.

The difference between the two sets is uncomputable. The shortest program for a task is not computable (Li and Vitányi, 2019), nontrivial semantic properties of programs are undecidable (Rice, 1953), and Blum’s speed-up theorem gives computable functions for which every algorithm has

an asymptotically faster one, so that no best program exists (Blum, 1967). Hutter (2002) constructs an algorithm asymptotically as fast as the fastest algorithm for any well-defined problem, with a constant that makes it unusable, which shows clearly that the in-principle answer and the resource-bounded answer differ.

Three frontiers are needed:

$$\mathcal{C}_t^{\text{deployed}} \subseteq \mathcal{C}_t^{\text{accessible}} \subseteq \mathcal{C}_t^{\text{known}} \subseteq \mathcal{C}^*.$$

A capability is known when an implementation has been described, accessible when the code, toolchain, parameters and expertise to run it can be obtained, and deployed when it runs on the device. The last results section concerns the first inclusion, which can reverse over time.

The closest precursor is Moor (1985), who described computers as logically malleable, able to be shaped to any activity characterisable through inputs, outputs and connecting logical operations. Malleability is the structural property; this paper describes how the reachable set changes under a budget over time. The objection that any physical process can be interpreted as computing anything is answered by the mechanistic account, in which a system computes in virtue of its causal organisation (Piccinini, 2007) and what makes something a computer depends on how its states are manipulated (Piccinini, 2008). The console answers prompts it has not seen, which no reinterpretation of an idle processor does.

Scope of the title

Wigner (1960) asked why mathematics developed for its own purposes applies so well to the physical world. Halevy, Norvig and Pereira (2009) borrowed his phrase for computation, arguing that useful behaviour can be obtained from data without a designed model, an approach that led to the trained parameters the console runs.

The analogy is weaker than Wigner's case. Wigner's mathematics was mostly developed without the physical application in view, so its fit is unexplained. Algorithms and representations are designed for computational substrates, so the fit between a quantisation scheme and a memory budget is expected, and extensibility follows from programmability, as Moor noted. Three aspects nonetheless need explanation, and the three results address them in turn: the magnitude of the gains, since a factor of 7.11 from representation and 3.2 from kernels are large; their timing, since the relevant techniques arrived two decades after the hardware; and the change in what the device is used for. *Unreasonable* here means unexpectedly large. A console with finite storage has a finite set of reachable configurations, and nothing here claims otherwise.

Requirements of the console runtime

This section is arithmetic on the runtime's published byte format with one free parameter. The scalar arithmetic rate is solved so that the full four-technique stack reproduces the reported throughput, which fixes it at 33.3 million multiply-accumulate pairs per second; every other value

below is a prediction from that calibration. The storage bandwidth is stipulated at 8 MB per second and varied in a sensitivity analysis.

Each of the four techniques has its own literature. Reduced-precision integer representation with per-block scales is the standard way to fit a network into a smaller budget without retraining it (Jacob et al., 2018; Frantar and Alistarh, 2022). Hand-written kernels for a specific vector unit are the kind of optimisation that superoptimisation automated, showing that the lowest layer of the software stack remains open to search (Massalin, 1987). Streaming and caching change what must be resident without changing the hardware. The arithmetic below shows which of these the capability depends on.

The block format stores 32 weights and one 16-bit scale in 18 bytes, or 0.5625 bytes per weight against 4 for full precision, a factor of 7.11. For 91.1 million parameters, the four representations require 347.5 MB at full precision, 173.8 MB at half precision, 92.3 MB at 8 bits with the same block scale, and 48.87 MB in the format used. Against the 48 MB the runtime can cache, these are 7.24, 3.62, 1.92 and 1.02 times the budget, so none fits.

Four-bit quantisation therefore reduces the model from about 7 times too large to about 2 percent too large, and the remaining 2 percent has to be handled otherwise. Adding the full-precision tensors gives a computed file of 49.37 MB against the reported 52 MB, a residual of 2.63 MB consistent with the embedded tokenizer, whose size the documentation does not state. The model of the artifact reproduces the reported file size to within 5 percent without using it as an input.

With 48 MB resident and 48.87 MB of weights, the runtime must read 0.87 MB from the card per token. The documentation, written independently of this analysis, states that the runtime “streams the last few MB from the memory stick per token”.

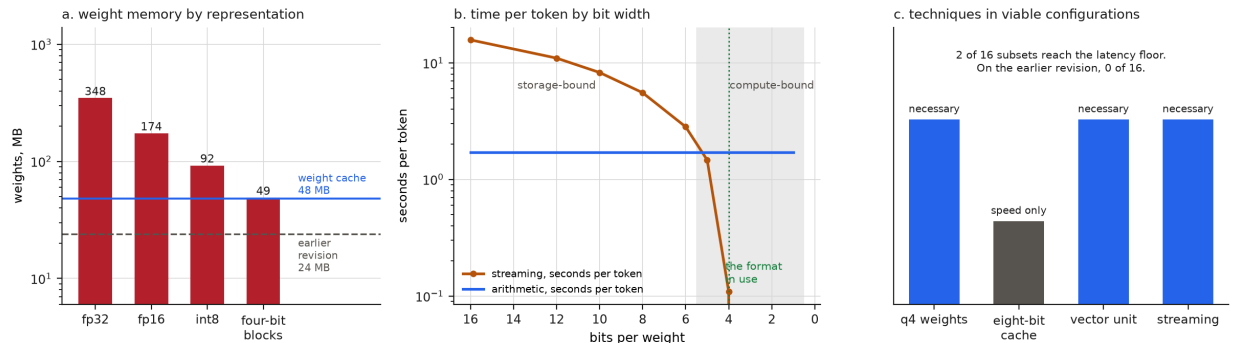


Figure 1: Memory and time requirements of the console runtime. (a) Weights in each representation against the 48 MB the console can cache and the 24 MB an earlier revision could; no representation fits outright. (b) Streaming time and arithmetic time per token as the representation narrows, with the region in which arithmetic dominates shaded and the format actually used marked. (c) Which of the four techniques appear in every configuration that reaches the latency floor.

The four techniques used by the runtime are four-bit weights, an 8-bit key-value cache, vector-unit kernels and streaming. Of the 16 subsets, two reach a floor of 0.4 tokens per second. The

minimal viable subset is four-bit weights with vector kernels and streaming, and each of these three is individually necessary, since removing any one leaves no viable configuration. The 8-bit cache is not necessary at this floor: it returns 3 MB to the weight cache and shortens the stream, and it becomes necessary only for floors above 0.456 tokens per second, the throughput of the three-technique subset. The distinction depends on bandwidth: at 4 MB per second instead of 8, only one subset reaches the floor and all four techniques are necessary.

On the 32 MB revision of the console, with the same 8 MB of runtime state, none of the 16 subsets reaches the floor, and the best configuration runs at 0.17 tokens per second. The same software and knowledge fail on the earlier hardware revision because the accumulated techniques cannot close the larger gap; the doubling of memory in the console's second revision is what made the capability reachable.

As the representation narrows, streaming time falls while arithmetic time does not, so the binding constraint changes. At or above 6 bits per weight the system is limited by storage bandwidth, and at or below 5 bits by arithmetic. The format in use has 4 bits, past this transition, so further compression gives almost nothing: a representation occupying no space at all would raise throughput from 0.55 to 0.585 tokens per second, a factor of 1.06. Further gains on this device must come from faster arithmetic.

Continuous improvement and discrete capability

The best available census of algorithmic progress is that of Sherry and Thompson (2021), who examined 57 textbooks and more than 1137 papers and identified 113 algorithm families with an average of eight algorithms each, 276 algorithms in total, an average of 1.44 improvements after the initial one per family. About half of the families improve little or not at all, and 14 percent of the remainder improve transformatively. At a problem size of one thousand, 18 percent of families improved faster than hardware; at one million, 30 percent; and at one billion, 43 percent. The median family improved by 6, 15 and 28 percent a year at those three sizes, and beat hardware only at problem sizes around a trillion. Leiserson et al. (2020) argue that performance must increasingly come from software, algorithms and architecture as transistor scaling slows.

Other measurements agree in direction and differ in magnitude, as expected for a heavy-tailed distribution sampled in different domains. Training a classifier to AlexNet-level ImageNet performance required 44 times fewer operations in 2019 than in 2012, an efficiency doubling every 16 months, against the 11-fold improvement Moore's law would have given (Hernandez and Brown, 2020). Compute-saving innovations in computer vision halve requirements about every nine months, with an interval from 4 to 25 (Erdil and Besiroglu, 2022). For language models, the compute needed to reach a fixed performance level halved about every 8 months, with an interval from 5 to 14 (Ho et al., 2024). In mathematical programming between 2001 and 2020, hardware improved about 20-fold while algorithms improved about 9-fold for linear programming and 50-fold for mixed-integer programming (Koch et al., 2022).

Koch et al. note that their factors “considerably underestimate the progress made on the algorithmic side: many problem instances can nowadays be solved within seconds, which the old codes are not able to solve within any reasonable time.” A speedup ratio cannot represent a change from infeasible to routine, and this change is what a budget crossing produces.

We model it by drawing 113 families from a three-component mixture whose parameters reproduce four figures from the census, giving a median rate of 0.145, a transformative share of 0.124 against a target of 0.14, and a share faster than hardware of 0.301. The hardware rate is not stipulated: taking it as the seventieth percentile of the mixture gives 0.446 a year, within the range of benchmark series (Moore, 2006 [1965]; Hennessy and Patterson, 2019; Denning and Lewis, 2017). Improvements arrive as a small number of jumps per family at the census rate of 1.44 after the first, consistent with the census finding that algorithms “experience large, but infrequent improvements”. Each task has two resource dimensions and a budget it must satisfy in both.

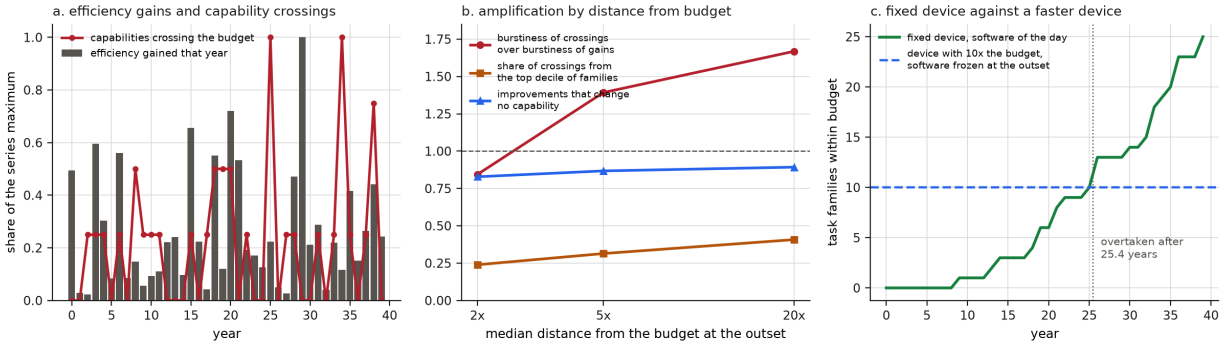


Figure 2: Efficiency gains and capability crossings. (a) Efficiency gained each year against capabilities crossing the budget each year, in one history at a median distance of 5 times budget. (b) Three measures against the distance from the budget at the outset: the burstiness of crossings relative to the burstiness of the gains, the concentration of crossings in the top decile of families, and the share of improvement events that change no capability. (c) The fixed device under improving software against a device with 10 times the budget under software frozen at the outset.

Most improvement events change no capability. Across the three regimes examined, between 0.828 and 0.893 of improvement events change no capability predicate, either because they improve a dimension that was not binding or because the gain falls short of the budget. This explains why a machine’s practical capability can remain unchanged for years while the literature reports steady progress.

Amplification depends on how far tasks start from their budget. The coefficient of variation of annual capability crossings, divided by the same statistic for the annual efficiency gains that produce them, is 0.84 when tasks start at 2 times their budget, 1.39 at 5 times and 1.67 at 20 times. At 2 times the budget crossings are smoother than gains, so there is no amplification. The share of crossings contributed by the top decile of families is 0.24, 0.31 and 0.41 across the three regimes, and the share of families that never cross in forty years rises from 0.51 to 0.69 to 0.76.

A comparison relevant to purchasing decisions sets the fixed device with current software against a device with 10 times its budget running software frozen at the starting date. In the two less demanding regimes the more powerful device is never overtaken, because a tenfold budget covers many tasks that were close to reach. In the most demanding regime it is overtaken after 25.4 years. Old hardware with new software outperforms new hardware with old software only on problems initially far out of reach for both.

Three considerations limit the generality of these results. No algorithm outperforms another averaged over all problems (Wolpert and Macready, 1997), so every claim here is specific to a task family and a distribution. Information processing has a physical lower bound on energy (Landauer, 1961). And which algorithms are discovered and refined depends on which hardware exists, so the separation of hardware and software used here is an experimental control (Hooker, 2021). The console model itself exists because other hardware was available to train it.

Known and deployable capability

Known capability accumulates, but deployment requires every component of a software stack to remain available. The console demonstration requires custom firmware, a cross-compilation toolchain, the runtime source, the quantised weight file and the tokenizer, and stops working if any one becomes unobtainable. Rothenberg (1995) noted that preserving data does not preserve the ability to use it, and later threat models for digital preservation address such compound dependencies (Rosenthal et al., 2005). Emulation can substitute partly for a surviving toolchain, with a fidelity that can be measured (Guttenbrunner and Rauber, 2012).

Give each component an annual hazard of becoming unrunnable and an annual rate of being re-stored. If a component is available with probability a , a stack of depth d is deployable with probability a^d , so deployable capability converges to a fraction of known capability. With a hazard of 0.03 a year and a revival rate of 0.12, a single component is available 0.80 of the time, and the stationary deployable fraction is 0.64, 0.512, 0.328, 0.168 and 0.069 at depths of 2, 3, 5, 8 and 12; by depth 4 less than half of known capability can be run. Simulating arrivals and failures gives deployable shares at forty years of 0.83, 0.70, 0.58, 0.43, 0.25 and 0.16 across depths 1, 2, 3, 5, 8 and 12. A five-component stack, the depth of the console demonstration, is at 0.43.

A steady flow of new artifacts masks the decay of old ones. When the flow of new artifacts declines, as it eventually does for every platform, deployable capability peaks and then falls. With interest decaying on a twelve-year scale, a five-component stack peaks in year 18, and a twelve-component stack peaks in year 6 and falls to 0.39 of its peak by year 60, although no knowledge is lost in the model.

Setting a stack's half-life equal to the mean interval between improvements in its field, 27.8 years given 1.44 improvements per family over four decades, requires a single component to survive a mean of 41 years and each component of a five-component stack to survive 201 years. Preserving a computational capability is therefore far more demanding than preserving a working device.

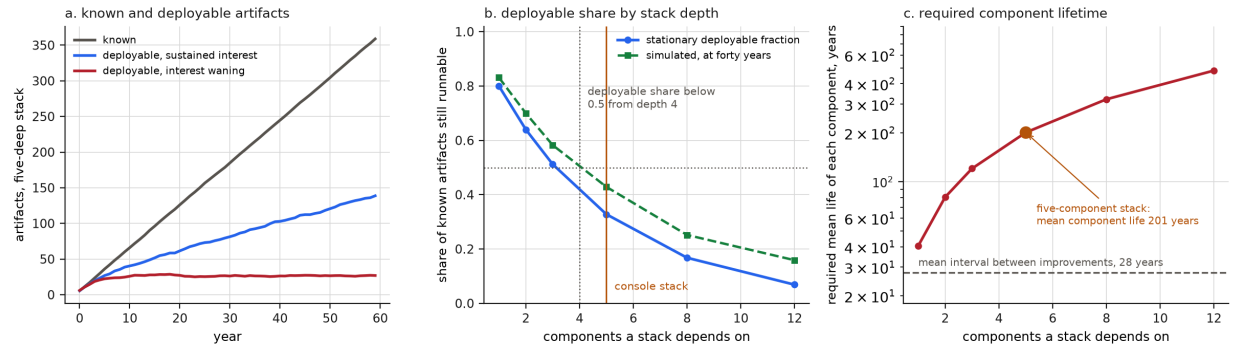


Figure 3: Known and deployable capability. (a) Known and deployable artifacts for a five-component stack, under sustained interest and under waning interest. (b) The share of known artifacts still runnable against the depth of the stack, in the stationary limit and in simulation at forty years. (c) The mean component life a stack of each depth would need for its half-life to match the interval between improvements in its field.

Limitations

The results do not imply that software gives hardware new physical capabilities. The set of physically reachable configurations is fixed; what changes is the set that can be reached within a budget with available knowledge.

The results are not a restatement of Turing universality, which concerns unbounded time and memory and does not address the 48 MB budget, the latency or the quality of the output, on which the console case depends.

The results do not suggest that the console became intelligent. The runtime’s author describes the model as “not really useful for anything”, and the case demonstrates a threshold crossing.

The results are not a general law of software progress. The census shows that half of algorithm families barely improve, the model reproduces this, and between 0.51 and 0.76 of families never cross a budget in forty years, depending on how far the budget lies.

The case does not show self-contained development. The parameters were trained elsewhere, on machines the console could not emulate in a human lifetime, and the cost of deploying a capability must be distinguished from the cost of discovering it. The console demonstrates extensibility of deployment; development cost per use falls only when amortised over many deployments, which is a separate claim.

The results do not show that old hardware is environmentally preferable. Energy per task may remain poor, and no lifecycle comparison is made. What the arithmetic supports is that obsolescence cannot be inferred from a device’s current software-defined function.

The separation of hardware and software is an experimental control, for the reasons given by Hooker.

Falsification tests

The first result is arithmetic and can be checked directly. If measurement on the hardware shows the runtime streaming much more or much less than 0.87 MB per token, or shows the system limited by storage bandwidth at four-bit weights, the model of the artifact is wrong. The ablation can also be run: the seven configurations the lattice predicts should fail should fail, and the two it predicts should run should reach the predicted throughput. Any configuration that runs when it should not falsifies the necessity claim. The prediction that no combination of these techniques runs on a PSP-1000 is the strongest and cheapest test.

The second result depends on a mixture calibrated to a census of exact algorithms, which excludes the approximate and heuristic methods where quantisation and distillation belong. A census that included them might show a different tail, and the calibration would change accordingly. The amplification result would be contradicted by a domain in which capability crossings are smoother than the efficiency gains producing them while tasks start far from their budget, and its dependence on distance from the budget would be contradicted by amplification in a domain where tasks start close to feasible.

The third result rests on a hazard and a revival rate that have not been measured. They can be measured by attempting to build and run a corpus of released artifacts of known dependency depth at regular intervals. If the hazard for well-maintained ecosystems is well below the 0.0246 a year that a single component would require, the decay result applies to neglected platforms and not to software in general. If it is above, deployable capability is declining in places that are not being measured.

A statement that a machine can perform a task is a statement about a machine, an artifact, a task, a budget, a quality threshold and a date. The result appears unreasonable only when the last four are omitted.

References

- Aaronson, S. (2013). Why philosophers should care about computational complexity. In B. J. Copeland, C. J. Posy, and O. Shagrir (eds.), *Computability: Turing, Gödel, Church, and Beyond*, 261–328. MIT Press.
- Blum, M. (1967). A machine-independent theory of the complexity of recursive functions. *Journal of the ACM*, 14(2), 322–336.
- Denning, P. J., and Lewis, T. G. (2017). Exponential laws of computing growth. *Communications of the ACM*, 60(1), 54–65.
- Erdil, E., and Besiroglu, T. (2022). Algorithmic progress in computer vision. arXiv:2212.05153.
- Frantar, E., and Alistarh, D. (2022). Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems*, 35, 4475–4488.

- Gu, A., and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752.
- Guttenbrunner, M., and Rauber, A. (2012). A measurement framework for evaluating emulators for digital preservation. *ACM Transactions on Information Systems*, 30(2), 1–28.
- Halevy, A., Norvig, P., and Pereira, F. (2009). The unreasonable effectiveness of data. *IEEE Intelligent Systems*, 24(2), 8–12.
- Hartmanis, J., and Stearns, R. E. (1965). On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117, 285–306.
- Hennessy, J. L., and Patterson, D. A. (2019). A new golden age for computer architecture. *Communications of the ACM*, 62(2), 48–60.
- Hernandez, D., and Brown, T. B. (2020). Measuring the algorithmic efficiency of neural networks. arXiv:2005.04305.
- Ho, A., Besiroglu, T., Erdil, E., Owen, D., Rahman, R., Guo, Z. C., Atkinson, D., Thompson, N., and Sevilla, J. (2024). Algorithmic progress in language models. arXiv:2403.05812.
- Hooker, S. (2021). The hardware lottery. *Communications of the ACM*, 64(12), 58–65.
- Hutter, M. (2002). The fastest and shortest algorithm for all well-defined problems. *International Journal of Foundations of Computer Science*, 13(3), 431–443.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–2713.
- Koch, T., Berthold, T., Pedersen, J., and Vanaret, C. (2022). Progress in mathematical programming solvers from 2001 to 2020. *EURO Journal on Computational Optimization*, 10, 100031.
- Landauer, R. (1961). Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3), 183–191.
- Leiserson, C. E., Thompson, N. C., Emer, J. S., Kuszmaul, B. C., Lampson, B. W., Sanchez, D., and Schardl, T. B. (2020). There’s plenty of room at the Top: What will drive computer performance after Moore’s law? *Science*, 368(6495), eaam9744.
- Li, M., and Vitányi, P. (2019). *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed. Springer.
- Massalin, H. (1987). Superoptimizer: A look at the smallest program. *ACM SIGARCH Computer Architecture News*, 15(5), 122–126.
- Moor, J. H. (1985). What is computer ethics? *Metaphilosophy*, 16(4), 266–275.
- Moore, G. E. (2006 [1965]). Cramming more components onto integrated circuits. *IEEE Solid-State Circuits Society Newsletter*, 11(3), 33–35.

- Piccinini, G. (2007). Computing mechanisms. *Philosophy of Science*, 74(4), 501–526.
- Piccinini, G. (2008). Computers. *Pacific Philosophical Quarterly*, 89(1), 32–73.
- Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2), 358–366.
- Rosenthal, D. S. H., Robertson, T., Lipkis, T., Reich, V., and Morabito, S. (2005). Requirements for digital preservation systems: A bottom-up approach. *D-Lib Magazine*, 11(11).
- Rothenberg, J. (1995). Ensuring the longevity of digital documents. *Scientific American*, 272(1), 42–47.
- Sherry, Y., and Thompson, N. C. (2021). How fast do algorithms improve? *Proceedings of the IEEE*, 109(11), 1768–1777.
- Technology Innovation Institute (2026). Falcon-H1-Tiny-90M-Instruct model card.
- Thatblend (2026). LLMPSP: A custom LLM runtime for the Sony PSP.
- Turing, A. M. (1937). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230–265.
- Wigner, E. P. (1960). The unreasonable effectiveness of mathematics in the natural sciences. *Communications on Pure and Applied Mathematics*, 13(1), 1–14.
- Wolpert, D. H., and Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82.